

Solving Differential Equations With LCHS and Extensions to LCU

Aadithya Iyer
SR No: 25828

Abstract

This work discusses a quantum simulation approach for the one-dimensional diffusion equation $\partial_t u = \alpha \partial_{xx} u$ using the Linear Combination of Hamiltonian Simulation (LCHS) framework combined with the Linear Combination of Unitaries (LCU) circuit construction. After discretising the spatial domain with $N = 16$ grid points with periodic boundary conditions, the semi-discrete system takes the form $\dot{u} = -Au$, where A is in our case, a time dependent matrix if α , the diffusion coefficient is time dependent. LCHS expresses e^{-AT} as a weighted integral of unitaries e^{-ikLT} . Numerical approximation is done via $J = 64$ composite Gauss–Legendre quadrature nodes. The resulting linear combination is encoded on a Quantum Circuit using Qiskit. We validate the method against a classical Forward-Time Centred-Space (FTCS) reference for first a constant ($\alpha = 0.25$) and attempt to extrapolate this to the time-dependent ($\alpha(t) = t$). Code can be found in the references section.

I. INTRODUCTION

Partial differential equations (PDEs) governing diffusion and heat transport are central to physics, chemistry, and many applications in engineering. Classical solvers scale polynomially with the number of degrees of freedom, restricting simulations with fine mesh sizes. Quantum computing offers the prospect of exponential compression through amplitude encoding: an N -point solution vector is stored in $\log_2 N$ qubits.

The diffusion is inherently *non-unitary* — the solution norm decays in time — which prevents direct representation as a quantum gate. The LCHS framework [1], [2] resolves this by decomposing the non-unitary propagator e^{-AT} into a continuous weighted sum of unitary Hamiltonian simulations. The LCU algorithm [3] then provides a quantum circuit (PREPARE–SELECT–PREPARE[†]) that implements this weighted sum, allowing us to actually implement the LCHS framework on a quantum computer.

This work implements and validates the full LCHS+LCU pipeline for the diffusion equation with both constant and time-dependent diffusivity, serving as a concrete worked example of quantum algorithms for non-unitary linear dynamics. The simulation is carried out using Qiskit’s statevector simulator [4] and validated against a classical FTCS reference.

II. BACKGROUND

A. Problem Formulation

We solve the one-dimensional diffusion equation on the periodic domain $x \in [0, 2\pi]$:

$$\frac{\partial u}{\partial t} = \alpha(t) \frac{\partial^2 u}{\partial x^2}, \quad u(x, 0) = u_0(x). \quad (1)$$

Discretising on $N = 16$ equally spaced grid points with spacing $\Delta x = 2\pi/N$ and periodic boundary conditions yields the linear ODE

$$\frac{du}{dt} = -Au, \quad A_{ij} = \frac{\alpha}{\Delta x^2} \begin{cases} 2, & i = j \\ -1, & |i - j| = 1 \pmod{N} \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

The formal solution is $u(T) = e^{-AT} u_0$. Since A is a positive semi-definite, the propagator is non-unitary.

B. LCHS Framework

For a matrix $A(t)$ decomposed as $A = L + iH$, where $L = (A + A^\dagger)/2$ (positive semi-definite part) and $H = (A - A^\dagger)/2i$ (Hermitian part), LCHS [2] provides the integral identity

$$\mathcal{T} e^{-\int_0^T A(s) ds} = \int_{\mathbb{R}} \frac{f(k)}{1 - ik} \mathcal{T} e^{-i \int_0^T (kL(s) + H(s)) ds} dk, \quad (3)$$

where $\mathcal{T}$ is the time-ordering operator and the kernel is

$$f(z) = \frac{1}{2\pi e^{-2\beta} e^{(1+iz)^\beta}}, \quad \beta \in (0, 1). \quad (4)$$

The parameter $\beta = 0.7$ is chosen for optimal numerical performance [2]. In our implementation the normalisation constant is $C_\beta = 2\pi e^{-2\beta}$.

For the diffusion equation $H = 0$, so each integrand reduces to the unitary $U(T, k) = e^{-ikLT}$. This is unitary because $-ikLT$ is Hermitian for real k and symmetric positive semi-definite L [9]. The practical kernel coefficient is

$$g(k) = \frac{1}{C_\beta(1 - ik) e^{(1+ik)^\beta}}. \quad (5)$$

C. Numerical Quadrature

The integral over $\mathbb{R}$ in Eq. (3) is truncated to $[-K, K]$ with $K = 4$ and approximated by composite Q -point Gauss-Legendre quadrature with panel width h_1 . With $Q = 4$ and $h_1 = 0.5$ this gives $K_{\text{panels}} = 2K/h_1 = 16$ panels and $J = K_{\text{panels}} \times Q = 64$ quadrature pairs $\{(c_j, k_j)\}_{j=1}^J$:

$$k_j = mh_1 + \frac{h_1}{2}(\xi_q + 1), \quad (6)$$

$$c_j = \frac{h_1}{2} w_q g(k_j), \quad (7)$$

where $\{\xi_q, w_q\}$ are the Gauss-Legendre nodes and weights on $[-1, 1]$ which have a specific numerical value at 4 quadrature points. Thus, and the approximation becomes

$$e^{-AT} \approx \sum_{j=1}^J c_j U(T, k_j). \quad (8)$$

Defining $\alpha_j = |c_j|$, $c_1 = \sum_j \alpha_j$, and phase $\phi_j = c_j/\alpha_j$, the coefficient factors as $c_j = \alpha_j \phi_j$ with ϕ_j absorbed into the unitary as $\tilde{U}_j = \phi_j U_j$ [3].

III. METHODOLOGY

A. LCU Circuit Construction

Following [3], the target operator is

$$T = \sum_{j=0}^{J-1} \alpha_j \tilde{U}_j. \quad (9)$$

The LCU circuit is built on $n_{\text{sys}} = \lceil \log_2 N \rceil = 4$ system qubits and $n_k = \lceil \log_2 J \rceil = 6$ ancilla qubits ($2^6 = 64 = J$), and proceeds in three stages.

Step 1 — Initial State Preparation. The system register is initialised to the normalised initial condition $\hat{u}_0 = u_0/\|u_0\|_2$, zero-padded to length $2^{n_{\text{sys}}}$, via Qiskit's `StatePreparation` [5].

Step 2 — PREPARE. A unitary V maps the ancilla ground state to a superposition encoding the quadrature weights [3]:

$$V|0^a\rangle = \frac{1}{\sqrt{c_1}} \sum_{j=0}^{J-1} \sqrt{\alpha_j} |j\rangle. \quad (10)$$

This is implemented via `StatePreparation`($\sqrt{\alpha_j/c_1}$) on the ancilla register.

Step 3 — SELECT. The block-diagonal unitary

$$\text{SEL} = \sum_{j=0}^{J-1} |j\rangle\langle j| \otimes U_j \in \mathbb{C}^{1024 \times 1024} \quad (11)$$

is constructed by placing each 16×16 block $\tilde{U}_j = \phi_j \cdot \exp(-ik_j LT)$ into the $(j \cdot N)$ -th diagonal block of an initially zero matrix, then applied as a `UnitaryGate` [6] acting on all qubits.

Step 4 — UNPREP. Applying $V^\dagger$ on the ancilla effects post-selection: when the ancilla measures $|0^a\rangle$, the system register holds $\frac{1}{c_1} \sum_j c_j \tilde{U}_j |\hat{u}_0\rangle \approx e^{-AT} |\hat{u}_0\rangle$. The physical solution is recovered as $u(T) = \text{Re}[\text{sv}[0 : N]] \cdot c_1 \cdot \|u_0\|_2$, where c_1 undoes the PREPARE normalisation and $\|u_0\|_2$ undoes the initial-state normalisation.

B. Extension to Time-Dependent $\alpha(t) = t$

In this case, the exponent $\int_0^T kL(s) ds$ no longer simplifies analytically. Each $\tilde{U}_j$ is approximated via the first-order Lie-Trotter formula [3], $e^{-it(H_1+H_2)} \approx e^{-itH_1} e^{-itH_2} + \mathcal{O}(t^2)$, incurring $\mathcal{O}(\delta t^2)$ error per slice. For $n_{\text{steps}} = 10$ slices of width $\delta t = T/n_{\text{steps}}$:

$$\tilde{U}_j \approx \phi_j \prod_{s=n_{\text{steps}}-1}^0 e^{-ik_j \delta t L(s \delta t)}$$

Here, each factor is computed via Qiskit's `PauliEvolutionGate` [8] with `LieTrotter(reps=1)` [7] synthesis. The structure of the previous unitaries, i.e the PREPARE and SELECT gate identical to the constant- α case.

Before proceeding to the experimental setup, we assume the Forward-Time Centred-Space scheme for the diffusion equation as the ground truth. With CFL parameter $r = \alpha \delta t / \Delta x^2 \leq 0.25$, the scheme is conditionally stable for $r \leq 0.5$.

IV. EXPERIMENTAL SETUP

All experiments use $N = 16$ grid points on $[0, 2\pi]$, $K = 4$, $h_1 = 0.5$, $Q = 4$ ($J = 64$), $\beta = 0.7$, and solution times $T \in \{0, 0.5, 1.0, 1.5, 2.0\}$ s. Three initial conditions are tested for the $\alpha = 0.25$ experiments:

$$u_0^{(1)} = \sin(x), \quad (12)$$

$$u_0^{(2)} = \sin(x) + \frac{1}{2} \sin(2x), \quad (13)$$

$$u_0^{(3)} = \sin(x) + \frac{1}{2} \sin(2x) + \frac{1}{3} \sin(3x). \quad (14)$$

For the time-dependent case $\alpha(t) = t$ we take $n_{\text{steps}} = 10$ Trotter slices. The FTCS CFL parameter is fixed at 0.25 to ensure numerical stability and we analyse for initial condition $u_0 = \sin(x)$.

V. RESULTS & DISCUSSION

A. Constant Diffusivity ($\alpha = 0.25$)

Thus, for our three initial conditions, we obtain four figures, showing the evolution of our three examples as well as the amplitude decay.

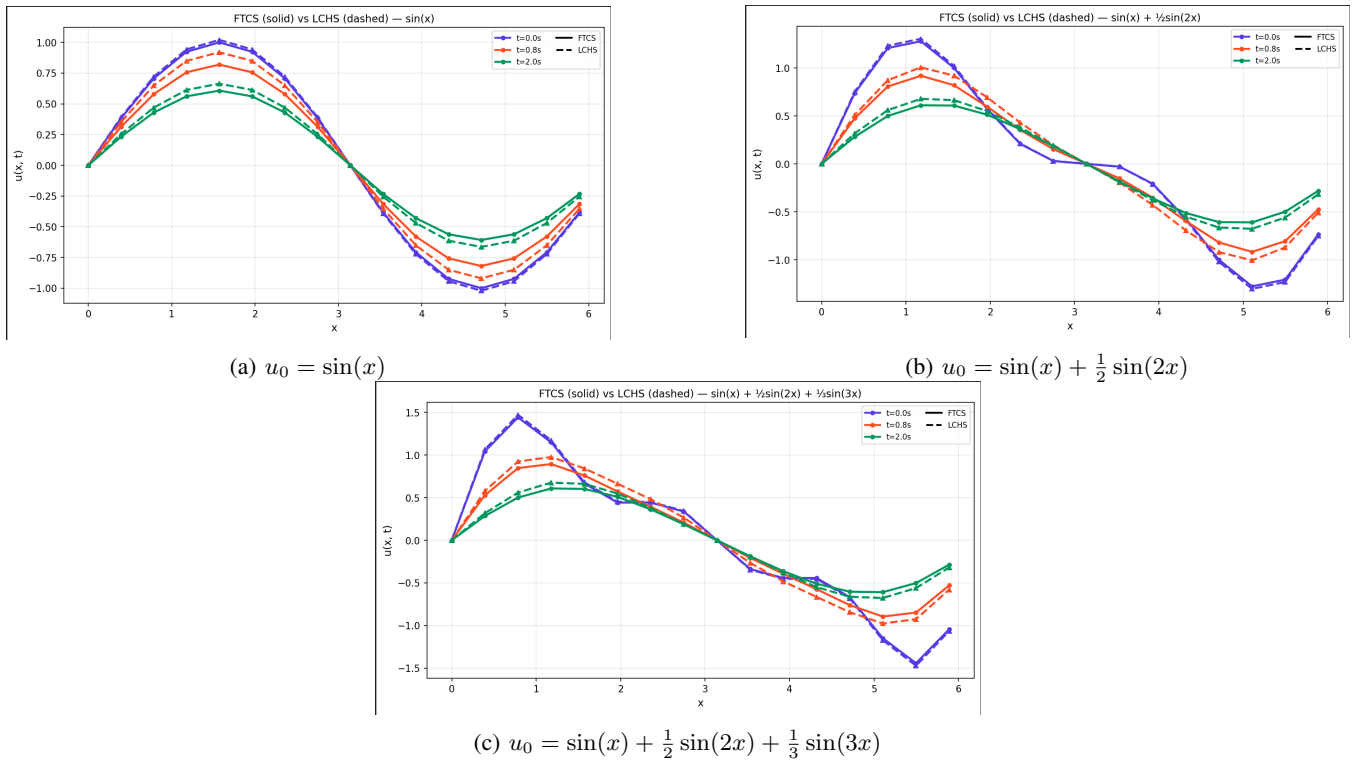


Fig. 1: FTCS (solid) vs LCHS (dashed) spatial evolution for all three initial conditions at five time snapshots, constant $\alpha = 0.25$.

Our amplitude decay curve shows that the LCHS magnitude is consistently higher than the FTCS reference across all three initial conditions. However, the gap between the FTCS and LCHS curves remains roughly constant throughout the simulation window. This confirms that the two methods decay at the same rate despite the amplitude offset. This behaviour is most clearly seen for $u_0^{(1)} = \sin(x)$, where both curves are nearly linear on this scale. For the multi-mode cases $u_0^{(2)}$ and $u_0^{(3)}$, the initial drop is steeper because the higher- k components decay faster before settling into the slower $k = 1$ dominated regime. This is confirmed via the spatial profiles: at each time snapshot the LCHS solution (dashed) lies uniformly above the FTCS solution (solid) in amplitude while preserving the correct waveform shape without any phase error oscillations.

1) Plot of Spectra

The one-sided Fast Fourier Transform (FFT) of the solution vector $u(x, t)$ at different times is now computed. This gives the normalised spectral amplitude $|\hat{u}(k)|/N$ at each physical wavenumber k . We then plot this as a heatmap over time, revealing which Fourier mode quickly decays. This provides a direct spectral-domain validation of both methods against the analytical prediction $|\hat{u}(k, t)| \propto e^{-\alpha k^2 t}$.

From Figure 3 we observe three active modes at $k = 1, 2, 3$ at $t = 0$, consistent with the initial condition. The $k = 3$ mode decays most rapidly, fading to black within the first 0.5s. Subsequently, $k = 2$ is dissipated by $t \approx 1.0$ s. The $k = 1$

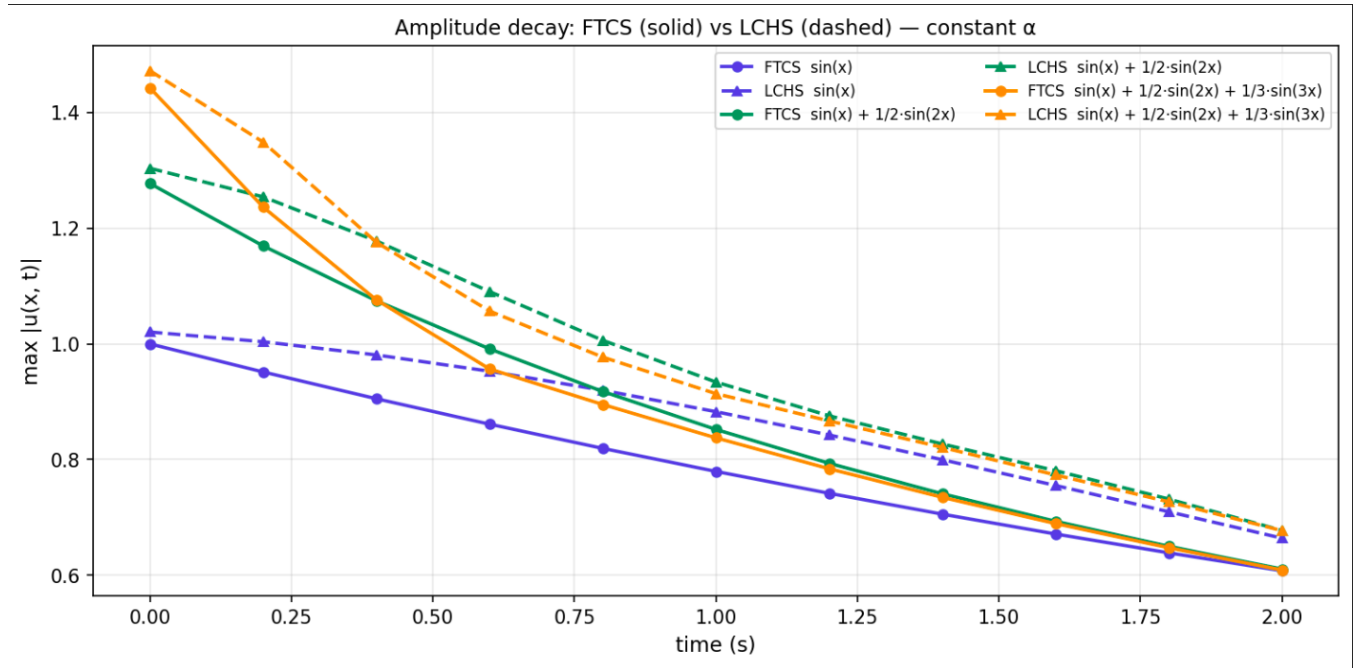
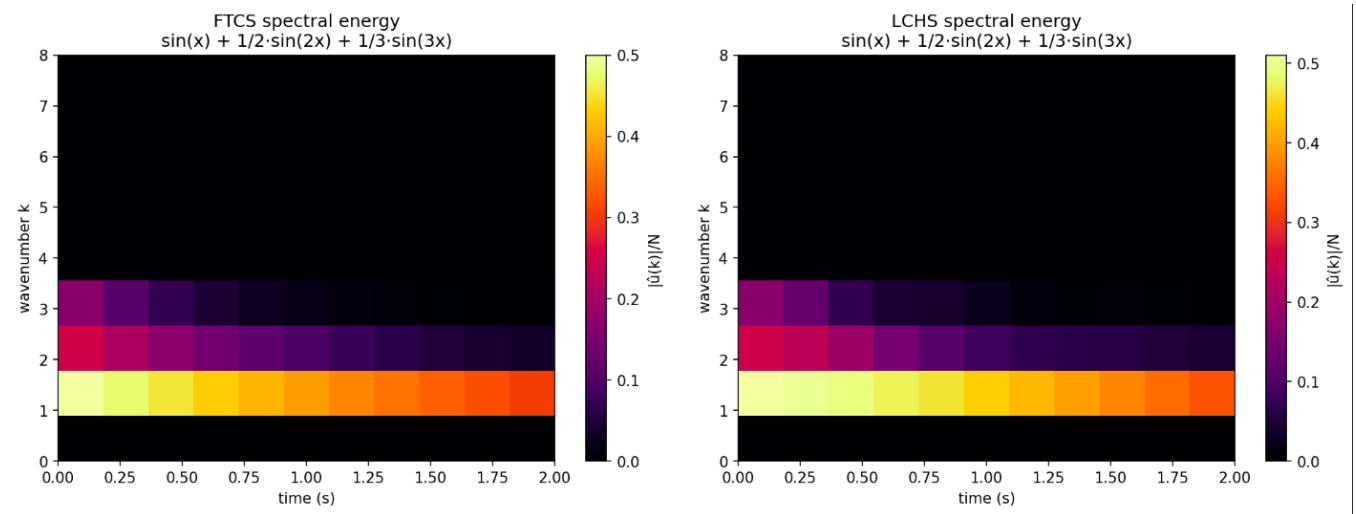


Fig. 2: Amplitude Decay for all 3 cases

Fig. 3: Spectral energy heatmap for $u_0 = \sin(x) + \frac{1}{2} \sin(2x) + \frac{1}{3} \sin(3x)$. Left: FTCS. Right: LCHS. Colour encodes normalised spectral amplitude.

mode persists throughout, decaying slowly from yellow at $t = 0$ to deep orange-red by $t = 2$ s. Crucially, the LCHS heatmap on the right is nearly indistinguishable from the FTCS reference on the left. The same three modes are present, decaying at the same rates. This confirms that the LCHS algorithm accurately captures the dissipative spectral dynamics of the diffusion equation.

B. Time-Dependent Diffusivity ($\alpha(t) = t$)

We now consider the case where the diffusivity grows linearly in time, $\alpha(t) = t$, applied to the initial condition $u_0 = \sin(x)$. Now, the operator is time-dependent, and the classical FTCS scheme must step through time using the instantaneous $\alpha(t_{\text{cur}})$ at each iteration while, LCHS encodes the integrated exponent exactly into a single matrix exponential. We do the same algorithm as discussed in the experimental setup section, taken from [3].

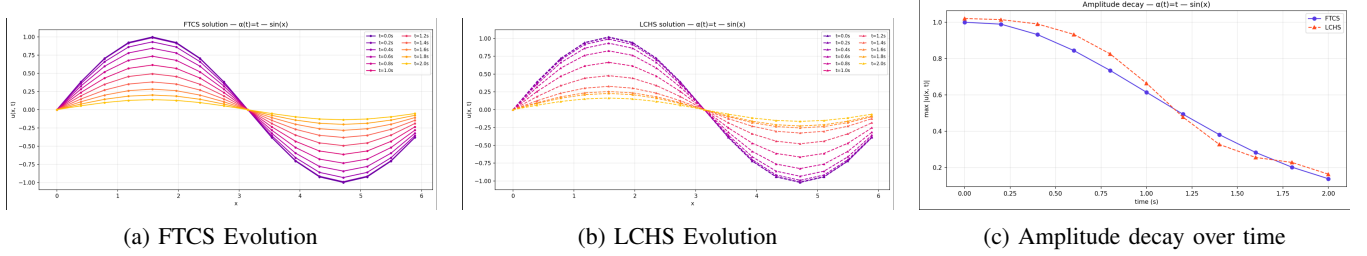


Fig. 4: FTCS and LCHS solutions for $u_0 = \sin(x)$ with $\alpha(t) = t$. Both methods show the characteristic slow initial decay followed by accelerating dissipation as $\alpha(t)$ grows.

A particularly interesting feature appears in the amplitude decay curve: the LCHS amplitude starts above FTCS as observed in the earlier case, but the two curves cross at approximately $t \approx 1.2$ s, after which LCHS and FTCS amplitudes start competing in terms of how quickly they decay. We also observe that both methods converge to near-zero amplitude by $t = 2$.

VI. CONCLUSION

This work implemented and validated the LCHS+LCU pipeline for the one-dimensional diffusion equation, benchmarked against a classical FTCS reference. Across all three initial conditions the algorithm correctly reproduces the dissipative dynamics, with per-mode decay rates in close agreement with the analytical prediction $e^{-\alpha k^2 t}$. The extension to time-dependent diffusivity $\alpha(t) = t$ was handled exactly by absorbing the integrated exponent $T^2/2$ directly into the matrix exponential, with no Trotterisation required.

An interesting observation across all experiments is the consistent positive amplitude offset of the LCHS solution relative to FTCS, the origin of which warrants further investigation. As a natural extension, incorporating the Hadamard test into the circuit would allow the solution to be extracted as expectation values, making the algorithm directly executable on real quantum hardware rather than a classical statevector simulator. Scaling the simulation to a larger number of qubits to handle finer spatial grids is also a straightforward and compelling next step.

REFERENCES

- [1] D. An, A. M. Childs, L. Lin, and L. Ying, “Linear combination of Hamiltonian simulation for nonunitary dynamics with optimal state preparation cost,” *arXiv preprint*, 2023.
- [2] D. An, J. Liu, and L. Lin, “Quantum algorithm for linear non-unitary dynamics with near-optimal dependence on all parameters,” *arXiv:2312.03916*, 2023. <https://arxiv.org/pdf/2312.03916>
- [3] L. Lin, *Lecture Notes on Quantum Algorithms for Scientific Computation*, UC Berkeley, 2022. (Sections 2: Amplitude Amplification; 6.1: Block Encoding; 7.3: LCU; Trotterisation.)
- [4] IBM Quantum, “Qiskit quantum_info.Statevector API,” https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.quantum_info.Statevector
- [5] IBM Quantum, “Qiskit circuit.library.StatePreparation API,” <https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.StatePreparation>
- [6] IBM Quantum, “Qiskit circuit.library.UnitaryGate API,” <https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.UnitaryGate>
- [7] IBM Quantum, “Qiskit synthesis.LieTrotter API,” <https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.synthesis.LieTrotter>
- [8] IBM Quantum, “Qiskit circuit.library.PauliEvolutionGate API,” <https://quantum.cloud.ibm.com/docs/en/api/qiskit/qiskit.circuit.library.PauliEvolutionGate>
- [9] Mathematics Stack Exchange, “If H is Hermitian, show that e^{iH} is unitary,” <https://math.stackexchange.com/questions/2731009>
- [10] Code for the final project uploaded onto Github: https://github.com/ExusBurn/SolvingLDE_QuantumComputing/tree/main